

DAY 16: DATA VISUALIZATION WITH SEABORN

Complete Beginner-Friendly Study Guide | 60-Day Data Science Learning Journey

Student: Muhammad Ahmad

Learning Goal

Learn Seaborn for statistical visualization, grouping with hue, distributions, outliers, correlations, and multi-variable exploratory analysis.

1. What is Seaborn?

Seaborn is a Python visualization library built on Matplotlib. It works especially well with Pandas DataFrames and makes many statistical charts simple to create.

```
import seaborn as sns
import matplotlib.pyplot as plt
```

Line by line: seaborn is imported as sns. Matplotlib pyplot is imported as plt for titles, labels and displaying figures.

2. Main Seaborn Charts

sns.lineplot() = trends. sns.barplot() = category comparison. sns.scatterplot() = relationship between numeric variables. sns.histplot() = distribution. sns.boxplot() = distribution and possible outliers. sns.countplot() = category counts. sns.heatmap() = matrix visualization. sns.pairplot() = relationships among several numeric variables.

3. Line Plot

```
sns.lineplot(data=df, x="Month", y="Sales")
plt.title("Monthly Sales")
plt.show()
```

data selects the DataFrame. x is the horizontal variable. y is the numeric variable. A line plot is useful for ordered or time-based trends.

4. Bar Plot

```
sns.barplot(data=df, x="Department", y="Marks")
plt.title("Average Marks by Department")
plt.show()
```

The categorical column is Department and the numeric column is Marks. Seaborn summarizes the numeric values for each category.

5. Scatter Plot

```
sns.scatterplot(data=df, x="Study_Hours", y="Marks")
plt.title("Study Hours vs Marks")
plt.show()
```

Each point represents an observation. It helps explore whether two numeric variables are associated.

6. Histogram

```
sns.histplot(data=df, x="Marks", bins=5, kde=True)
plt.title("Marks Distribution")
plt.show()
```

bins controls the intervals. kde=True adds a smooth estimated distribution curve.

7. Box Plot

```
sns.boxplot(data=df, x="Department", y="Marks")
plt.title("Marks by Department")
plt.show()
```

A box plot summarizes a distribution using quartiles and can highlight observations that may be unusually far from the main distribution.

8. Count Plot

```
sns.countplot(data=df, x="Department")
plt.title("Students by Department")
plt.show()
```

countplot counts observations in each category.

9. Correlation Heatmap

```
correlation = df[["Marks", "CGPA", "Study_Hours"]].corr()
sns.heatmap(correlation, annot=True)
plt.title("Correlation Heatmap")
plt.show()
```

corr() calculates pairwise correlation values. heatmap displays the matrix. annot=True writes values inside the cells.

10. Pair Plot

```
sns.pairplot(df[["Marks", "CGPA", "Study_Hours"]])
plt.show()
```

pairplot creates multiple pairwise views and is useful during Exploratory Data Analysis.

11. hue, style and size

```
sns.scatterplot(
    data=df,
    x="Study_Hours",
    y="Marks",
    hue="Department"
)
```

hue separates groups visually. style can change marker style by category. size can map a variable to marker size.

12. Complete Student Dataset

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data = {
    "Name": ["Ali", "Usman", "Hamza", "Ahmad", "Bilal", "Hassan", "Zain", "Omar"],
    "Department": ["CS", "CS", "SE", "SE", "CS", "SE", "CS", "SE"],
    "Marks": [85, 72, 91, 78, 65, 88, 95, 70],
    "CGPA": [3.2, 3.0, 3.6, 3.1, 2.8, 3.4, 3.8, 2.9],
    "Study_Hours": [5, 4, 7, 5, 3, 6, 8, 4]
}

df = pd.DataFrame(data)
```

The dataset contains student names, departments, marks, CGPA and study hours.

13. Full Student Performance Analysis Project

```
print("Average Marks:", df["Marks"].mean())
print("Average CGPA:", df["CGPA"].mean())
print("Highest Marks:", df["Marks"].max())
print("Top Student:")
print(df.loc[df["Marks"].idxmax()])

sns.countplot(data=df, x="Department")
plt.title("Students by Department")
plt.show()

sns.barplot(data=df, x="Department", y="Marks")
plt.title("Average Marks by Department")
plt.show()

sns.histplot(data=df, x="Marks", bins=5, kde=True)
plt.title("Marks Distribution")
plt.show()

sns.scatterplot(data=df, x="Marks", y="CGPA", hue="Department")
plt.title("Marks vs CGPA")
plt.show()

sns.scatterplot(data=df, x="Study_Hours", y="Marks", hue="Department")
plt.title("Study Hours vs Marks")
plt.show()

sns.boxplot(data=df, x="Department", y="Marks")
plt.title("Marks Distribution by Department")
plt.show()

correlation = df[["Marks", "CGPA", "Study_Hours"]].corr()
print(correlation)
sns.heatmap(correlation, annot=True)
plt.title("Correlation Heatmap")
plt.show()

sns.pairplot(df[["Marks", "CGPA", "Study_Hours"]])
plt.show()
```

Expected key results: average marks = 80.5, average CGPA = 3.225, highest marks = 95, top student = Zain.

14. Interpretation

The count plot compares department sizes. The bar plot compares average marks by department. The histogram shows the distribution of marks. Scatter plots explore relationships between marks, CGPA and study hours. The box plot compares distributions. The heatmap summarizes correlations. A visible association or correlation does not automatically prove causation.

15. Practice Tasks

1. Bar plot for Ali=80, Usman=75, Hamza=95, Ahmad=88.
2. Line plot for monthly sales.
3. Scatter plot of Study Hours vs Marks.

4. Histogram with bins=5.
5. Count plot for CS, SE and AI.
6. Box plot of marks.
7. Box plot comparing marks by department.
8. Correlation matrix and heatmap for Marks, CGPA and Study_Hours.
9. Pair plot for Marks, CGPA and Study_Hours.
10. Complete Student Performance Visualization Dashboard using countplot, barplot, histogram, scatterplot, boxplot, heatmap and pairplot.

16. Complete Task Solutions

```
# 1
names=["Ali", "Usman", "Hamza", "Ahmad"]
marks=[80, 75, 95, 88]
sns.barplot(x=names, y=marks)
plt.show()

# 2
months=["Jan", "Feb", "Mar", "Apr"]
sales=[100, 120, 150, 140]
sns.lineplot(x=months, y=sales)
plt.show()

# 3
sns.scatterplot(data=df, x="Study_Hours", y="Marks")
plt.show()

# 4
sns.histplot(data=df, x="Marks", bins=5)
plt.show()

# 5
temp=pd.DataFrame({"Department":["CS", "CS", "SE", "AI", "CS", "SE"]})
sns.countplot(data=temp, x="Department")
plt.show()

# 6
sns.boxplot(y=df["Marks"])
plt.show()

# 7
sns.boxplot(data=df, x="Department", y="Marks")
plt.show()

# 8
correlation=df[["Marks", "CGPA", "Study_Hours"]].corr()
sns.heatmap(correlation, annot=True)
plt.show()

# 9
sns.pairplot(df[["Marks", "CGPA", "Study_Hours"]])
plt.show()

# 10
# Combine the charts from the complete project above.
```

17. Common Mistakes

- Using column names that do not exist.
- Forgetting data=df when using DataFrame columns.
- Confusing countplot with barplot.
- Treating correlation as proof of causation.
- Making pairplots with too many variables.
- Assuming KDE represents raw observations.
- Calling plt.show() before completing chart settings.

18. Quick Revision

Seaborn provides Data Science-friendly statistical charts. Use lineplot for trends, barplot for category comparisons, scatterplot for relationships, histplot for distributions, boxplot for distributions and possible outliers, countplot for category counts, heatmap for matrices and pairplot for multiple numeric relationships. Use hue to compare groups.

19. Final Checklist

- Import Seaborn and Matplotlib
- Create line, bar and scatter plots
- Create histograms and box plots
- Use countplot
- Use hue

- Calculate and visualize correlation
- Use heatmap
- Use pairplot
- Combine Pandas and Seaborn
- Understand correlation vs causation

20. Learning Path

Python → NumPy → Pandas → Data Cleaning → Data Visualization → Statistics → Machine Learning → Deep Learning